

Senior

TEAM NAME

ROBONIUM BANGLADESH

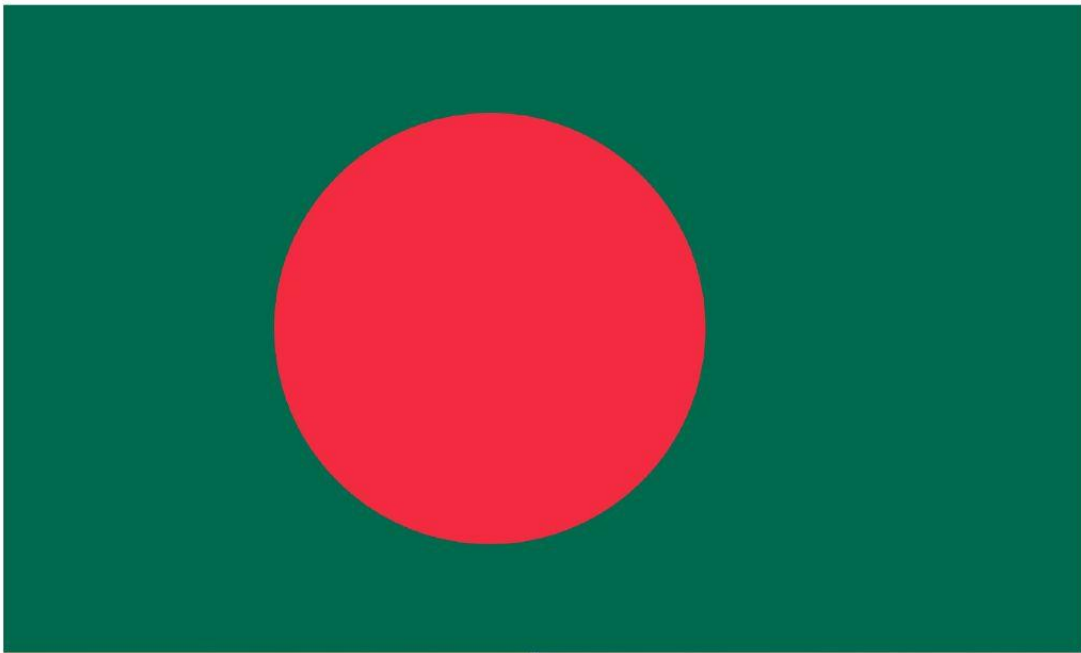


Table of Contents

1. Team Presentation.....	3
2. Summary of Project Idea:.....	4
3. Problem Statement:.....	4
4. Presenting Robotic Solution:.....	6
5. Key Features:.....	6
6. Availability of Similar Concepts:.....	7
7. Evolution of Project Idea:.....	8
8. Mechanical Structure:.....	9
9. Circuit Diagram:.....	11
10.Flow Charts:.....	12
11. Coding Explanation:.....	13
12.Robot Operation:.....	16
13.One Tried Practical Use Case:.....	17
14.Challenges we faced:.....	17
15.Our Robot Mission:.....	17
16.Social Impact:.....	18
17.Entrepreneurship Aspects:.....	19
18.Future Scopes and Improvements:.....	20
19.List of Sources:.....	21

Team Presentation



Israfil Shaheen Arannya

Class 11, Cantonment Public School & College, Rangpur

Roles: Machine Learning, Python Coding (AI Part), 3D Modeling, Hardware Construction, Documentation



Quazi Mostahid Labib

Class 11, Sunnydale School, Dhaka

Roles: JRC Board Coding, RPI Setup, Python Library Installation, Hardware Construction, Documentation



Tafsir Tahrir

Class 11, Sunnydale School, Dhaka

Roles: RPI Setup, Python Library Installation, 3D Modeling, Hardware Construction, Documentation

Summary of Project Idea:

In an era marked by technological advancements, the wall climbing robot for ship hull inspection emerges as a pioneering innovation aimed at revolutionizing maritime maintenance and safety. This robotic system represents a convergence of robotics and machine learning, meticulously designed to scale the vertical surfaces of ship hulls, detect structural flaws, and enhance the overall integrity of maritime vessels.

By combining cutting-edge technology with a commitment to safety, the wall climbing robot introduces a transformative approach to ship hull inspection, significantly reducing human exposure to hazardous environments while optimizing the efficiency and accuracy of assessments. This innovation beckons a future where maritime maintenance is characterized by enhanced safety, efficiency, and reliability, ultimately ensuring the longevity and resilience of our seafaring vessels.

Problem Statement:

The maritime industry, vital for global commerce and transportation, faces persistent challenges in the maintenance and inspection of ships. In a service period of a ship, it has to perform dry-docking in each 4-5 years. During dry docking inspectors perform a full ship hull inspection and look for cracks, scratches and other structural flaws. Traditional inspection methods require ISO-certified inspectors to operate in challenging environments, leading to:

Safety risks: Human inspectors conducting hull inspections are exposed to hazardous conditions, including reaching heights to get a better view of ship surface and not having a proper way to inspect the sloping hull. Ensuring the safety of them is crucial in maritime operations.



612 x 466

High operational time and costs: Balancing the need for comprehensive inspections with cost-effectiveness is a constant challenge for shipowners and operators. Reducing operational costs without compromising the quality of inspections is essential.

Potential inaccuracies in assessment: Accurate detection of structural flaws, cracks, and defects in ship hulls is critical to prevent potential accidents, environmental damage, and costly repairs. Existing methods may lack the precision needed for early detection. These factors reduce efficiency and safety for vessels traveling long distances in the oceans.

Presenting Robotic Solution:



Team Robonium Bangladesh presents an extraordinary solution to overcome the hull inspection problem, the **Hull Ascender**. A robot capable of climbing vertical surfaces and detecting possible cracks, holes and paint decays in the ship's hull using machine learning.

We are using special kind of **Magnetic Wheels** that we have designed to make the robot climb walls.

The Hull Ascender

Key Features:

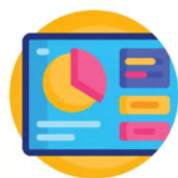
Here are the key features of **Hull Ascender**:



Magnetic Wheels For Climbing Ship Hull



Detecting Flaws On Hull Using ML



Reporting Inspection Data On Dashboard



Ground Controls & Communication

Availability of Similar Concepts:

Through our research, we found various wall-climbing devices used in the marine sector for cleaning ship hulls or measuring hull thickness, such as the Sparrow, which is a prototype. Moreover we found out ShipTest which is a robotic inspection system designed for welding inspection and corrosion mapping on ship hulls. It uses laser weld tracking technology and automated data processing for efficient assessments. However, our robot provides a better solution to identify any cracks in ship hulls that can lead to potential dangers while operating the ship. It uses AI to detect cracks, scratches and other structural flaws. The versatile climbing mechanism made it unique.

Here is a comparison chart of Sparrow, ShipTest and The Hull Ascender:

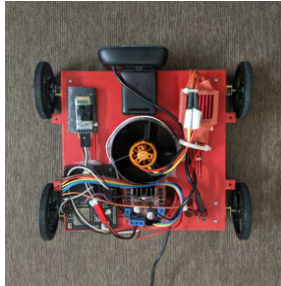
Feature	ShipTest	Sparrow	Hull Ascender	Benefits of Hull Ascender
Inspection Type	Limited to Welding inspection, corrosion mapping	Metal thickness inspection	Wide range of inspection types including detection of cracks, corrosion, paint issues, welding faults, scratches.	Specialized in almost all kinds of inspection types.
Deployment	Difficult to deploy, requires more personnel to set up.	Required to be deployed on surfaces and not so manoeuvrable.	Easy Deployment, can climb the ship surface using special magnetic wheels.	Unique climbing capability for thorough inspections.
Detection Technology	Laser weld tracking system	Accurate thickness assessment	Machine learning and image classification	Advanced machine learning for accurate detection and provides more versatile possibilities
Efficiency	Data being collected for assessment by specialists.	Only can be used for thickness assessment	Real-time detection and accurate outputs reducing time and cost.	Faster identification of critical issues in a cheaper and more accurate way.

Evolution of Project Idea:

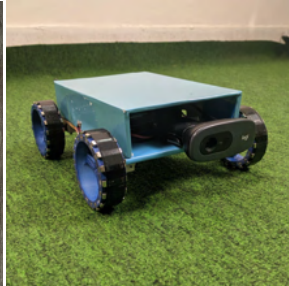
While implementing our idea in real life, we have tried different approaches. Making the robot climb a vertical wall was the most challenging thing to do.



V1



V2



V3



V4

Version 01: In the very first version of the robot we decided to test how suction powered climbing mechanism works. We used a 4500KV Brushless motor to create suction and make the robot climb. The result was not that satisfactory. We were using rigid and thick wheels. The robot was too heavy for the suction to work properly. Then we thought about making the robot lightweight.

Version 02: To address the weight issue, we created a lightweight 3D printed robot body and switched to thinner wheels to increase pressure. While this initially showed promise, we had to remove many components, leaving only the base. This made the robot lighter and capable of climbing vertical walls, but it sacrificed maneuverability, restricting it to forward and backward movements, and the power-hungry suction motor wasn't practical for real-world use.

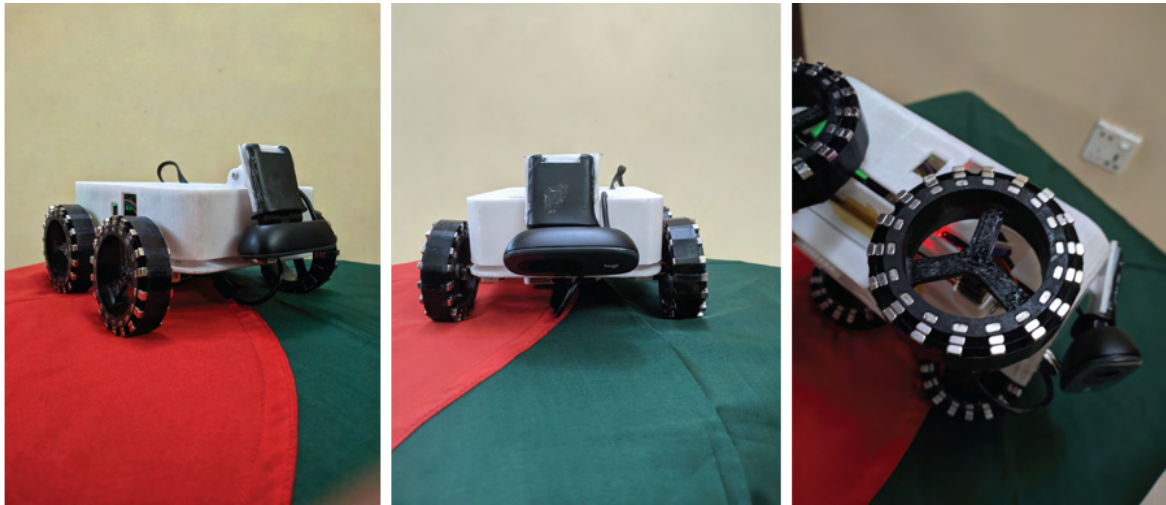
Version 03: After finding out all the dependencies and limitations of the suction method, we thought of trying a different one. We completely moved to magnets. As the Ship Hull of a ship is made with steel. Magnets will work perfectly. So we designed a special kind of wheel. We covered the wheel with 2 layers of magnets so that while it gets in touch with the vertical surface, it stays on it using the traction. This method removes the weight dependency and increases the maneuverability to a new extent. With this design approach we will be able to make the robot fully autonomous. Also this is less power consuming.

Version 04 (Current): In the Version 03, our Machine Learning Unit, the Raspberry Pi 4 was in the ground because of weight dependency. We had to extend the camera cable so that we could connect it to the Computer Unit. But as we managed to remove the weight dependency, we installed everything of the robot on its actual body. We made a totally new 3D Printed Body for visuals and aesthetics. But it seemed even after using magnets it had some weight issues. So we added a 3rd layer of magnet. We also improved our machine learning model in this version. We added 2 more classes in it. We also designed a Graphical User Interface for Post-inspection Report.

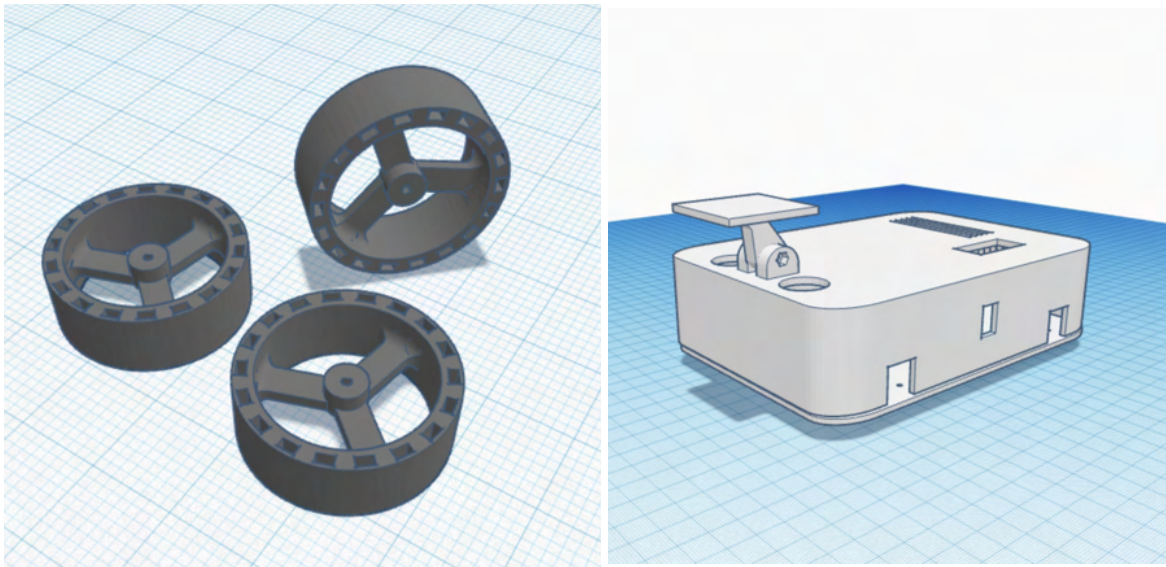
Mechanical Structure:

We have used the **JRC Board (Made in Bangladesh)** as the brain for the navigation system. Moreover, our custom wheels have been used for the climbing mechanism and L298N Motor Driver has been used to control the robot's driving system. The driving system is being controlled using Bluetooth from a ground station. Besides, we have used the Raspberry Pi Model 4B for our self trained machine learning feature.

Hull Ascender from different views:



The Custom Wheels and Robot Body 3D Model:



We have designed Special Custom 3D Printed Wheels where we put magnets to make the mechanism work. We covered the outer part of the wheel with 2 layers of **20mmX5mmX3mm sized Neodymium Magnets**. We have also added a layer on the inner part of the wheel.



Wheel Specifications:

- Outer Diameter: 65 mm
- Inner Diameter: 48 mm
- Thickness: 20 mm
- Magnets per wheel: 54
- Material: PLC Filament

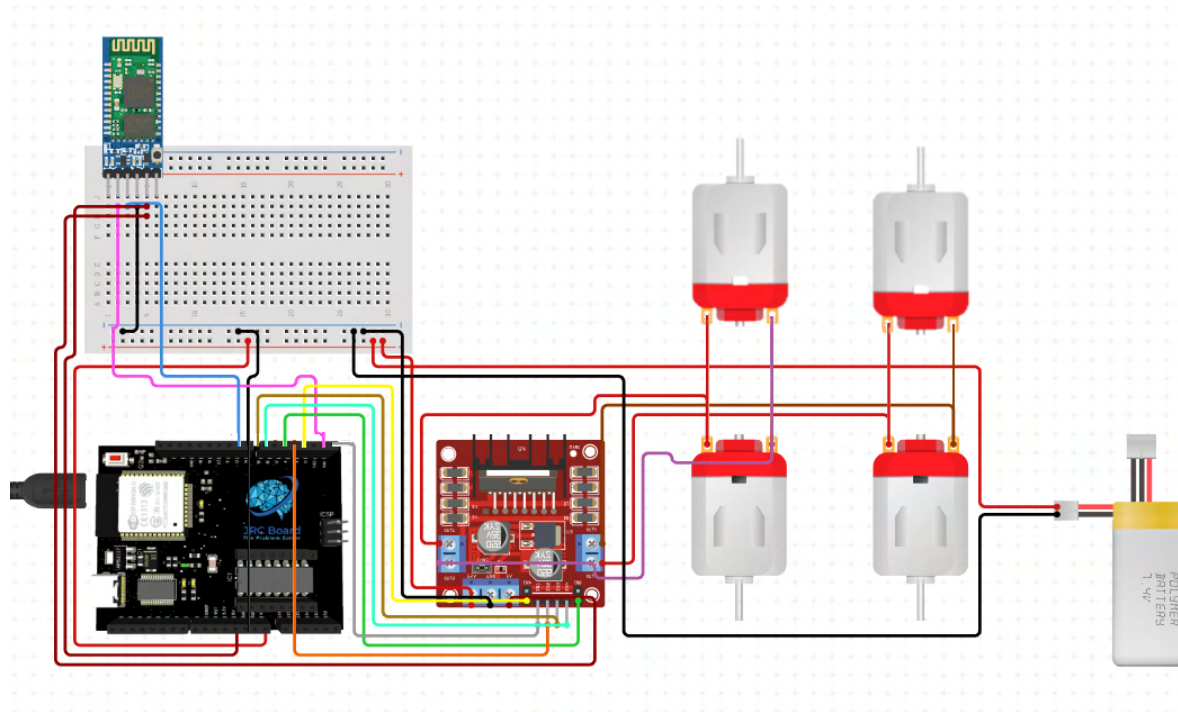
With a total of 4 wheels the robot can strongly hold its position on a vertical Steel Wall.



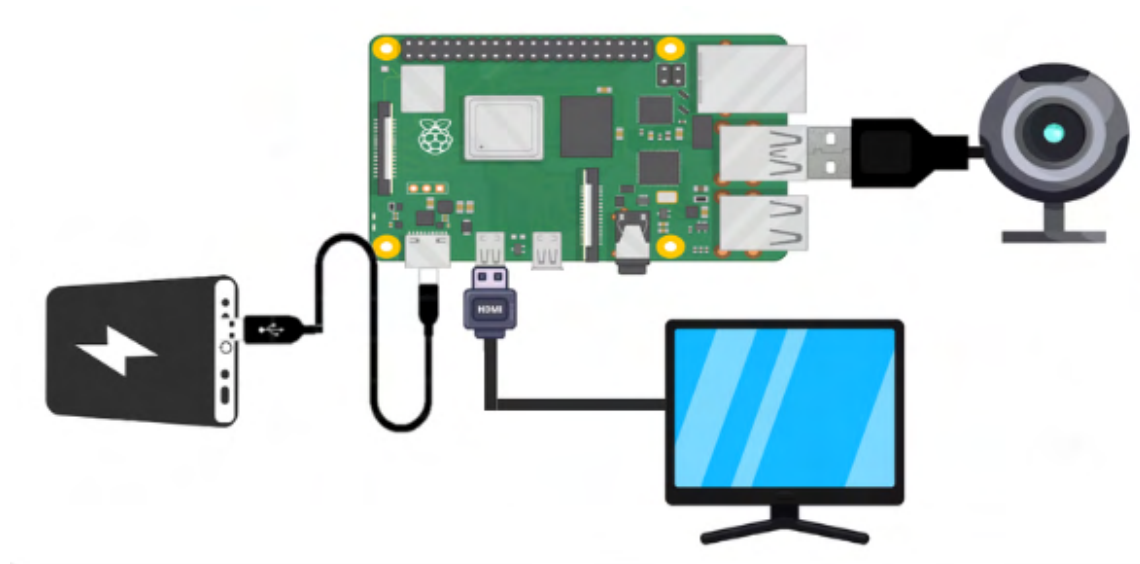
Hull Ascender Holding its position on a Ship Hull

Circuit Diagram:

Robot Unit:

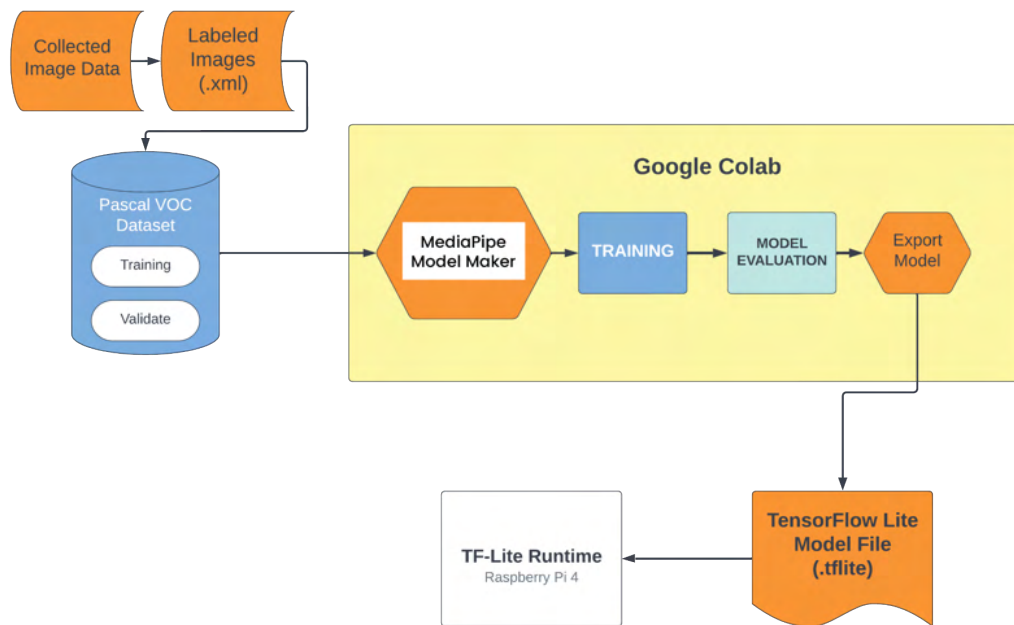


RPI Unit:

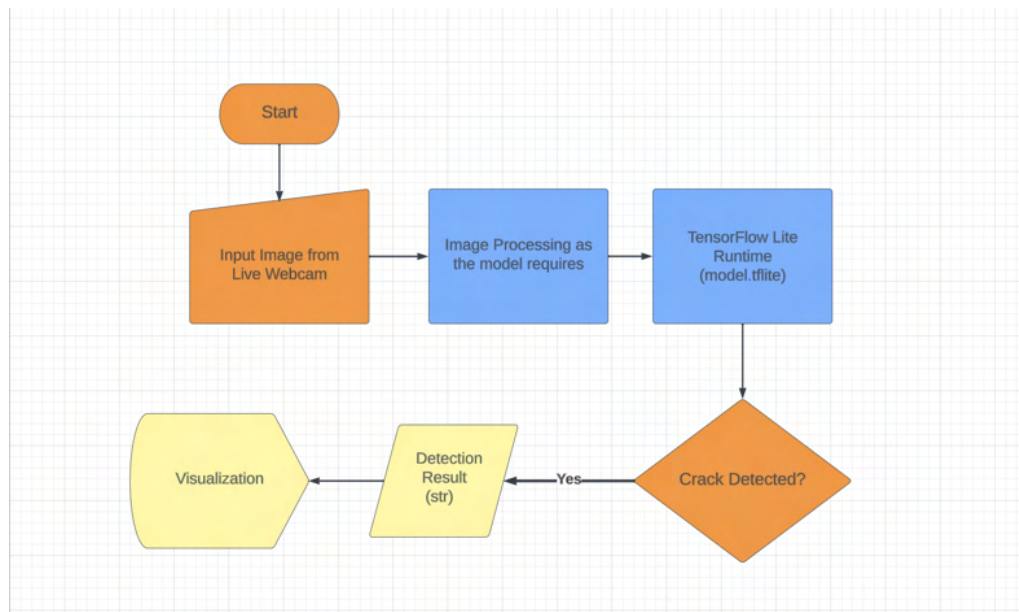


Flow Charts:

Model:



Crack Detection Algorithm:



Coding Explanation:

Disclaimer: Only the special code snippets have been shown here. Find the full code in the GitHub Repository.

GitHub Repository Link:

<https://github.com/Robonium-Bangladesh/Hull-Ascender.git>

Robot Navigation:

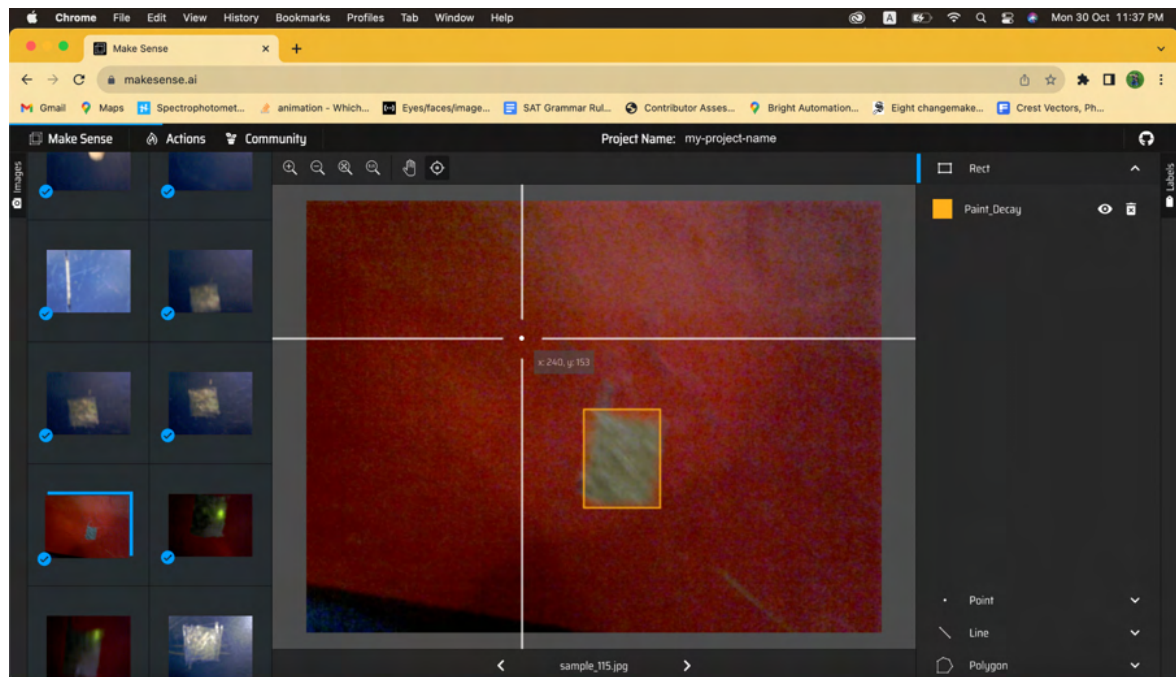
We have used a **JRC Board (Made in Bangladesh)** to control the movement of our robot, which is operated using a wireless remote controller to navigate the robot as required.

Machine Learning:

A Tensorflow Lite Model has been trained to detect cracks on ship hulls using Mediapipe Model Maker. The process was done in the following steps:

1. Creating a custom dataset in Pascal VOC Format containing sample hull crack images.
2. Labeling the images in 2 classes.
3. Use the custom dataset to train the model in Google Colab using Mediapipe Model Maker.
4. Creating an algorithm which will use the tflite model and detect cracks.

Model:



Labeling the images and creating dataset

```

+ Code + Text Copy to Drive
[12] Total params: 2579263 (9.84 MB)
Trainable params: 2533631 (9.67 MB)
Non-trainable params: 45632 (178.25 KB)

Epoch 1/50
/usr/local/lib/python3.10/dist-packages/keras/src/backend.py:452: UserWarning: `tf.keras.backend.set_learning_phase` is
warnings.warn(
WARNING:tensorflow:Gradients do not exist for variables ['conv2dbn_block_1/batch_normalization/gamma:0', 'conv2dbn_block_1/batch_normalization/gamma:0', 'conv2dbn_block_1/batch_normalization/gamma:0', 'conv2dbn_block_1/batch_normalization/gamma:0', 'conv2dbn_block_1/batch_normalization/gamma:0', 'conv2dbn_block_1/batch_normalization/gamma:0']
18/18 [=====] - 70s 652ms/step - total_loss: 12.7852 - cls_loss: 12.4802 - box_loss: 0.0050 - mod
Epoch 2/50
18/18 [=====] - 5s 271ms/step - total_loss: 1.2668 - cls_loss: 1.0547 - box_loss: 0.0031 - mod
Epoch 3/50
18/18 [=====] - 5s 254ms/step - total_loss: 0.9573 - cls_loss: 0.7699 - box_loss: 0.0026 - mod
Epoch 4/50
18/18 [=====] - 6s 306ms/step - total_loss: 0.6973 - cls_loss: 0.5404 - box_loss: 0.0020 - mod
Epoch 5/50
18/18 [=====] - 5s 251ms/step - total_loss: 0.5443 - cls_loss: 0.3992 - box_loss: 0.0018 - mod
Epoch 6/50
18/18 [=====] - 4s 234ms/step - total_loss: 0.4444 - cls_loss: 0.3126 - box_loss: 0.0015 - mod
Epoch 7/50
18/18 [=====] - 6s 312ms/step - total_loss: 0.3651 - cls_loss: 0.2461 - box_loss: 0.0013 - mod
Epoch 8/50
18/18 [=====] - 4s 238ms/step - total_loss: 0.3370 - cls_loss: 0.2207 - box_loss: 0.0012 - mod
Epoch 9/50
18/18 [=====] - 7s 367ms/step - total_loss: 0.2873 - cls_loss: 0.1783 - box_loss: 0.0011 - mod
Epoch 10/50
18/18 [=====] - 5s 255ms/step - total_loss: 0.2894 - cls_loss: 0.1749 - box_loss: 0.0012 - mod
Epoch 11/50
18/18 [=====] - 7s 371ms/step - total_loss: 0.3306 - cls_loss: 0.1765 - box_loss: 0.0020 - mod

```

Training

```

[13] loss, coco_metrics = model.evaluate(val_data, batch_size=4)
print(f"Validation loss: {loss}")
print(f"Validation coco metrics: {coco_metrics}")

38/38 [=====] - 3s 50ms/step - total_loss: 0.3860 - cls_loss: 0.2519 - box_loss: 0.0016 - model_
creating index...
index created!
creating index...
index created!
Running per image evaluation...
Evaluate annotation type *bbox*
DONE (t=0.80s).
Accumulating evaluation results...
DONE (t=0.18s).
Average Precision (AP) @[ IoU=0.50:0.95 | area= all | maxDets=100 ] = 0.478
Average Precision (AP) @[ IoU=0.50 | area= all | maxDets=100 ] = 0.759
Average Precision (AP) @[ IoU=0.75 | area= all | maxDets=100 ] = 0.524
Average Precision (AP) @[ IoU=0.50:0.95 | area= small | maxDets=100 ] = 0.122
Average Precision (AP) @[ IoU=0.50:0.95 | area=medium | maxDets=100 ] = 0.529
Average Precision (AP) @[ IoU=0.50:0.95 | area= large | maxDets=100 ] = 0.409
Average Recall (AR) @[ IoU=0.50:0.95 | area= all | maxDets= 1 ] = 0.550
Average Recall (AR) @[ IoU=0.50:0.95 | area= all | maxDets= 10 ] = 0.594
Average Recall (AR) @[ IoU=0.50:0.95 | area= all | maxDets=100 ] = 0.594
Average Recall (AR) @[ IoU=0.50:0.95 | area= small | maxDets=100 ] = 0.181
Average Recall (AR) @[ IoU=0.50:0.95 | area=medium | maxDets=100 ] = 0.656
Average Recall (AR) @[ IoU=0.50:0.95 | area= large | maxDets=100 ] = 0.802
Validation loss: [0.3859929144382477, 0.2518972158432007, 0.001581852207891643, 0.330989807844162]
Validation coco metrics: {'AP': 0.47781506, 'AP50': 0.7589679, 'AP75': 0.524245, 'APs': 0.12203562, 'APm': 0.52947414, 'A

```

Validating

```

model.export_model('CracksModel.tflite')
!ls exported_model
files.download('exported_model/CracksModel.tflite')

Exporting a floating point model
/usr/local/lib/python3.10/dist-packages/keras/src/engine/functional.py:639: UserWarning: Input dict contained keys ['6']
inputs = self.flatten_to_reference_inputs(inputs)
WARNING:tensorflow:Skipping full serialization of Keras layer <official.vision.modeling.retinanet_model.RetinaNetModel ob
WARNING:tensorflow:Skipping full serialization of Keras layer <keras.src.layers.merging.add.Add object at 0x7e34f8144730>
WARNING:tensorflow:Skipping full serialization of Keras layer <keras.src.layers.merging.add.Add object at 0x7e34f8261810>
WARNING:tensorflow:Skipping full serialization of Keras layer <keras.src.layers.merging.add.Add object at 0x7e34f824c1c0>
WARNING:tensorflow:Skipping full serialization of Keras layer <keras.src.layers.merging.add.Add object at 0x7e34f8175d80>
WARNING:tensorflow:Skipping full serialization of Keras layer <keras.src.layers.merging.add.Add object at 0x7e34f8275030>
WARNING:tensorflow:Skipping full serialization of Keras layer <keras.src.layers.merging.add.Add object at 0x7e34e099ab60>
WARNING:tensorflow:Skipping full serialization of Keras layer <keras.src.layers.merging.add.Add object at 0x7e34e09ff370>
WARNING:tensorflow:Skipping full serialization of Keras layer <official.vision.modeling.layers.detection_generator.Multil

```

Exporting Model

Algorithm:

Here is the part of our code which detects the cracks using the model.

```
101 # Continuously capture images from the camera and run inferenc
102
103 while cap.isOpened():
104     success, image = cap.read()
105     if not success:
106         sys.exit(
107             'ERROR: Unable to read from webcam. Please verify your webcam settings.'
108         )
109
110     image = cv2.flip(image, 1)
111
112     # Convert the image from BGR to RGB as required by the TFLite model.
113     rgb_image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
114     mp_image = mp.Image(image_format=mp.ImageFormat.SRGB, data=rgb_image)
115
116     # Run object detection using the model.
117     detector.detect_async(mp_image, time.time_ns() // 1_000_000)
118
119     # Show the FPS
120     fps_text = 'FPS = {:.1f}'.format(FPS)
121     text_location = (left_margin, row_size)
122     current_frame = image
123     cv2.putText(current_frame, fps_text, text_location, cv2.FONT_HERSHEY_DUPLEX,
124                 font_size, text_color, font_thickness, cv2.LINE_AA)
125
126     if detection_result_list:
127         #print(detection_result_list)
128         current_frame, has_detections = visualize(current_frame, detection_result_list[0])
129         if has_detections == True:
130             detected_category = detection_result_list[0].detections[0].categories[0].category_name
131             if detected_category == "Crack":
132                 Crack_count += 1
133                 cv2.imwrite(f'{folder}/{detected_category}_{Crack_count}.jpg', current_frame)
134             elif detected_category == "Hole":
135                 Hole_count += 1
136                 cv2.imwrite(f'{folder}/{detected_category}_{Hole_count}.jpg', current_frame)
137             elif detected_category == "Paint_Decay":
138                 Paint_Decay_count += 1
139                 cv2.imwrite(f'{folder}/{detected_category}_{Paint_Decay_count}.jpg', current_frame)
140
141
142
```

Robot Operation:

Robot Navigation: The navigation is controlled using the JRC Board, which takes the ground commands and sends them to the motor driver. The navigation system is quite simple and straightforward.

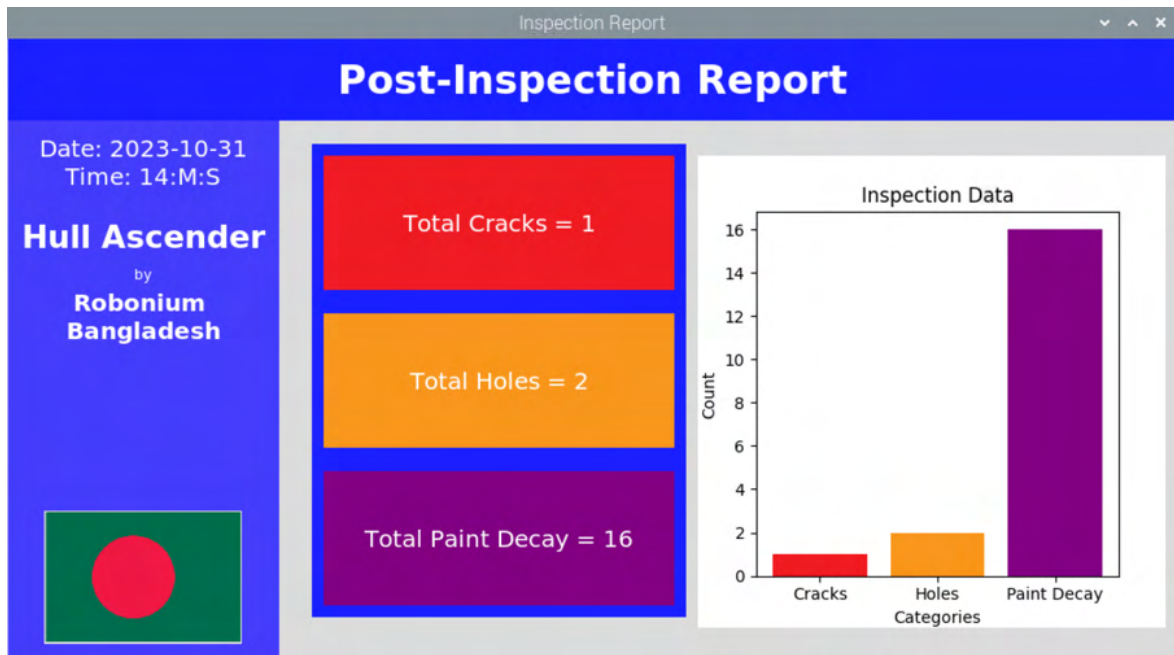
Machine Learning:

Our machine learning is run using Raspberry Pi 4B. We are running TFLite runtime in it. The process of crack detection is as follows:

- A webcam continuously surveys the hull of the ship and processes the frame according to the model requirements
- The frame is then transferred to the model
- If a crack is detected, it is classified
- An image of the crack is captured for reference while repairing
- Gives a Post-Inspection Report on a Dashboard showing number of cracks, holes and paint decays.

GUI Dashboard:

Our algorithm can detect 3 types of structural flaws. Cracks, Holes and Paint Decays. We created a Graphical User Interface to show the operator report of the inspection it did. This Dashboard Shows how many Cracks, Holes or Paint Decays it has detected as well as a Graph Chart for better understanding.



Our Dashboard Showing Inspection Report

One Tried Practical Use Case:

Firstly, we will start up the robot by connecting the battery. It will enable all controls. Then we will deploy the robot on a ship hull. Then the robot can be moved in any direction with the ground controls. In the meantime, the machine learning algorithm will start its work and continuously check for cracks, holes or paint decays on the surface. If it finds any, then it will have a count on and store it in a database folder for future reference. Then after the operator stops the system, it will show an inspection report on a Dashboard.

Challenges we faced:

As our robot has enhanced features on both hardware and software, we had to face ample challenges in both places.

1. In the 2nd version making the robot lightweight to climb walls was difficult.
2. Also in V2 finding the perfect speed for the Brushless Motor took a lot of tries.
3. To make the robot move on vertical surfaces we had to find the perfect motors which are not so heavy but also give the torque we require.
4. Creating the best possible design of robot wheels to ensure enough grip as well as proper movement on any metallic surface.
5. Finding the most suitable type of magnet to fit our requirements
6. Making the robot as compact as possible.
7. Finding ML libraries that support Raspberry Pi and its OS.
8. As the robot will be moving so we had to make the Machine Learning model as fast and efficient as possible.
9. Getting all the components installed inside the robot not affecting the overall balance and exceeding weight limit.
10. Tuning the robot to get the perfect speed as well as turn radius.

Our Robot Mission:

Our robot falls in the segment **Connecting Over Water** of the robot mission. It helps shipping become more efficient and safe, as only one robot can inspect a huge amount of area of the ship's hull in quick time and provide accurate results as to whether there are any cracks present or not, which significantly reduces cost, time and inaccuracy that would have occurred if conducted manually.

Social Impact:

There are various ways in which our robot will have a positive impact on society.

- **Improved Safety for Workers:** By replacing or complementing ISO-certified human inspectors with robots for hull inspections, you reduce the risks associated with sending people into hazardous environments. This can lead to fewer accidents, injuries, and loss of life among maritime workers.
- **Reduced Maintenance Costs:** Ship owners and operators can benefit from reduced maintenance costs. Early detection of cracks and defects allows for timely repairs, preventing more extensive and expensive damage. This cost savings can translate into more affordable shipping and transportation, which can positively impact various industries and consumers.
- **Increased Efficiency and Productivity:** The efficiency and speed of inspections performed by robots can lead to increased productivity in the maritime sector. Ships spend less time in dry docks or off-schedule for inspections and repairs, contributing to more reliable transportation and global trade.
- **Global Trade and Economic Impact:** Reliable and efficient ship inspections can have a positive impact on global trade. Reduced downtime for inspections and maintenance can help keep goods flowing smoothly around the world, benefitting economies and consumers alike.

Entrepreneurship Aspects:

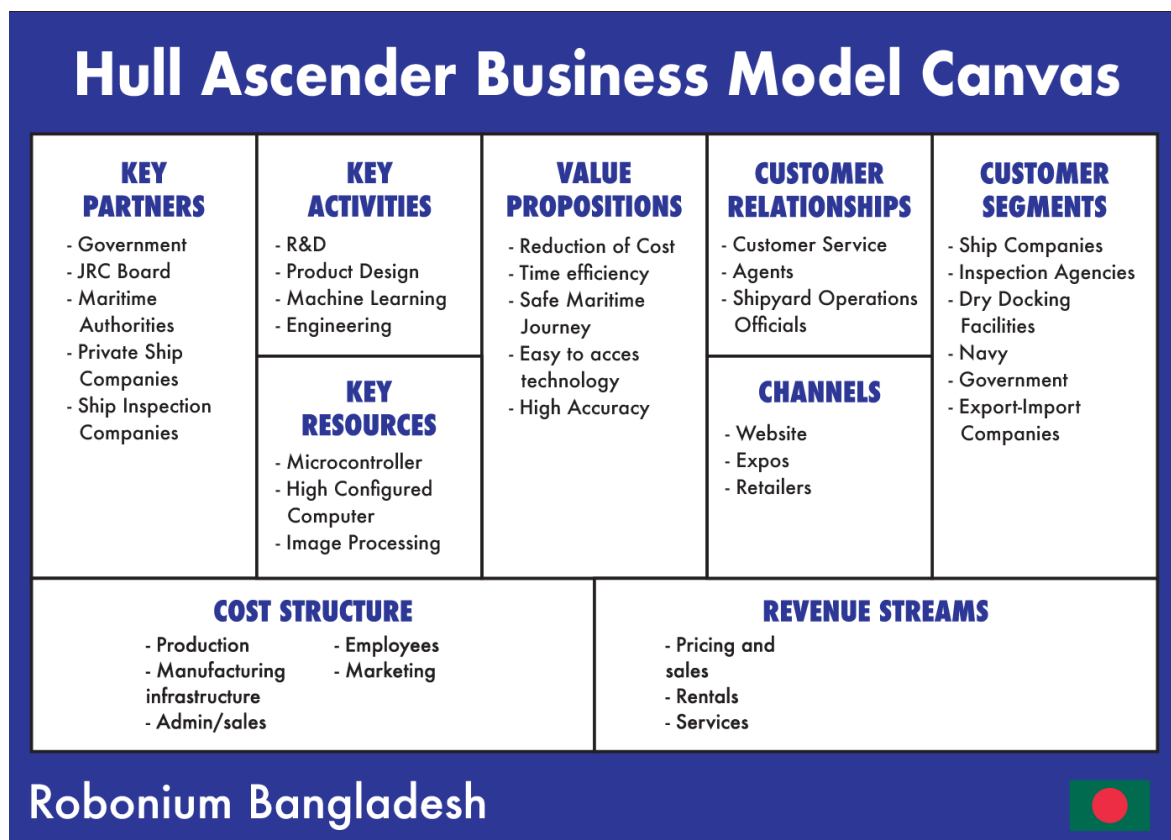
Vision:

Hull Ascender is a prototype of the evolutionary product that we are working on. Our goal is to fully optimize this system and make a perfect incorporation of advanced mechanical design and artificial intelligence to make it easier for ship companies to make sure their ships are safe in the oceans.

Demand:

In the world of around **50,000 to 60,000** Cargo Ships this product can be a game changer in the field of ship inspection and maintenance. Ship Inspectors in the US make around **50,000\$ to 77,000\$** per year. In this high demand market, our robot can be launched at a very reasonable price. Many of the companies would want to cut their inspection costs using this product.

Here is the business model of **Hull Ascender** –



Future Scopes and Improvements:

The Hull Ascender is a prototype of the extraordinary product we are dreaming of. We are working on this robot. After a ton of updates and upgrades we achieved our primary goals and made the robot climb a vertical steel wall quite efficiently. Our machine learning model is enriched with modular data samples. Our robot is highly scalable. More tuning and adjustments can take it to the next level.

Here are a few things that's under development now:

- **Autonomous:** We are working to make the robot fully autonomous so that after deployment it can automatically inspect the whole ship without the need of external controls.
- **Enhanced ML Model:** We are working to make the model enriched with more classes and a variety of data. It will be able to detect any kind of structural flaws on the Ship.
- **Micro Cracks:** With special kinds of sensors and measuring devices we plan to get on inspecting micro cracks which are not visible on a camera with Hull Ascender.
- **Adding More Camera:** By adding more cameras we can look into even narrower spaces. It will increase the robot's performance and we will have the ability to detect a wider range of flaws.

List of Sources:

1. Google Developers
2. marineinsight.com
3. Comparably.com
4. shiptest.eu
5. researchgate.net

We discussed with:



Arif Hoque

Naval Architect and Marine Engineer
Bangladesh University of Engineering & Technology, Class of 2007



Quazi Mortuza Ali

Senior Vice President, Bank Asia
MBA, Brac University, Class of 2012

Thank You